

ГЛУБОКИЙ АУДИТ ПРОЕКТА

ai-os-new

Ревью исходного кода, инфраструктуры, документации и тестирования

Репозиторий: <https://github.com/n95887174-source/ai-os-new/>

Дата аудита: 16 июня 2026 | Аудитор: Super Z AI (senior reviewer, QA engineer)

1. КРАТКИЙ ОБЩИЙ ВЫВОД

Проведён глубокий аудит репозитория ai-os-new (SuperAgents OS v4.5.0) -- крупного React + TypeScript приложения, реализующего концепцию "AI OS" с маршрутизацией интеллектуальных запросов, системой дебатов, управлением API-ключами и развитой наблюдаемостью. Аудит охватил четыре области: исходный код (src/), инфраструктуру и DevOps, документацию и спецификации, а также тестовую инфраструктуру.

В ходе аудита обнаружено **106 проблем**, из которых **14 критических**, **28 высоких**, **39 средних** и **25 низких**. Наиболее серьёзные системные проблемы включают: resolver, который молча проглатывает ошибки при обращении к неинициализированным сервисам; утечку API-ключей в консоль браузера; $O(n*m)$ сложность при потоковой передаче сообщений; критический баг в entrypt.sh, из-за которого Docker-контейнер не запускается; сломанный CI-пайплайн; и фундаментальную нехватку тестового покрытия (~6.4% файлов покрыты тестами). Документация содержит многочисленные противоречия: разные документы указывают разные версии, разное количество агентов, разное количество панелей и контрактов, а SINGLE_SOURCE.md -- предполагаемый единый источник истины -- содержит заведомо неверные утверждения об отсутствии циклических зависимостей и инлайн-стилей.

Area	Critical	High	Medium	Low	Total
Исходный код (src/)	4	10	11	6	31
Инфраструктура/DevOps	2	5	10	6	23
Документация/Спецификации	4	8	10	8	30

Тестирование/QA	4	5	8	5	22
Total	14	28	39	25	106

2. КРИТИЧЕСКИЕ ПРОБЛЕМЫ (Critical)

2.1 Исходный код

C-CODE-01

Critical

Расположение: kernel/resolver.ts (строки 22-43)

Суть ошибки: Функция resolve() возвращает Proxy, который бросает исключения при обращении к свойствам неинициализированных сервисов. Метод getInstance() молча возвращает null, и последующее обращение к свойствам null приводит к необработанным ошибкам. Fallback-заглушки предусмотрены только для settingsService и keyService -- остальные 40+ сервисов будут вызывать крах приложения при обращении до завершения runtime.start(). Модульный код в stores (useKeyStore строка 64, debateLiveStore -- подписки на события на уровне модуля) выполняется немедленно при импорте.

Риск: Белый экран при инициализации приложения, состояние гонки между регистрацией сервисов и первым обращением. Пользователи увидят крах приложения до загрузки.

Рекомендация: Предоставить безопасные fallback-заглушки для BCeX резолвящихся сервисов, либо изменить Proxy-обработчик так, чтобы он возвращал по-ор функции вместо бросания исключений. Добавить try/catch в Proxy get handler с логированием предупреждения вместо бросания ошибки.

C-CODE-02

Critical

Расположение: stores/useKeyStore.ts (строки 144, 149), kernel/events/event-bus.ts (строка 316)

Суть ошибки: События KEYS_LOADED и KEY_UPDATED передают полный массив ApiKey[], содержащий фактические строки API-ключей. В режиме разработки EventBus логирует полные payloads через console.debug, что означает, что значения API-ключей записываются в консоль браузера. Строка console.log в useKeyStore логирует count, но переменная updatedKeys содержит полные данные ключей.

Риск: Утечка учётных данных через DevTools браузера, системы отчётов об ошибках или при демонстрации экрана. Любое расширение браузера может получить доступ к консоли.

Рекомендация: 1) Редактировать значения API-ключей во всех EventBus DEV-mode логах. 2) Не передавать полные строки ключей через event payloads -- использовать только ID ключей. 3) Добавить поле redacted в тип ApiKey для отображения. 4) Удалить console.log в production-сборках.

C-CODE-03

Critical

Расположение: stores/useChatStore.ts (строки 443-617), useKeyStore.ts (строки 139-232), debateLiveStore.ts (строки 42-102)

Суть ошибки: Stores подписываются на события на уровне модуля. Обработчики useChatStore итерируют по sessions > history > responses при КАЖДОМ чанке потока. С 200 сообщениями и 5 ответами это 1000 итераций на чанк. Поточная передача генерирует десятки чанков в секунду, создавая O(n) нагрузку на

CPU. Метод `destroy()` в `debateLiveStore` и `topologyTraceStore` существует, но никогда не вызывается.

Риск: Подтормаживание UI при потоковой передаче, рост памяти от замыканий, удерживающих снимки состояния, потенциальный OOM при длительных сессиях.

Рекомендация: 1) Использовать индексированный поиск (Map по `requestId`) вместо сканирования массивов. 2) Батчить stream chunks с `requestAnimationFrame`. 3) Вызывать `destroy()` при размонтировании компонентов. 4) Рассмотреть `subscribeWithSelector` Zustand.

C-CODE-04

Critical

Расположение: `stores/useChatStore.ts` (строки 176-268)

Суть ошибки: Функция `sendMessage` проверяет `get().isSending` и выходит, если `true`, затем устанавливает `isSending: true`. Между проверкой и установкой другой вызов может пройти, т.к. `get()` и `set()` не атомарны. Кроме того, если `emit` вызывает синхронный ответ (mock-адаптер), обработчик ответа сбрасывает `isSending` до завершения текущего `sendMessage`.

Риск: Дублирование сообщений, застрявшее состояние UI (`isSending: true` навсегда), некорректная привязка ответов к запросам.

Рекомендация: Использовать Set активных request ID вместо булева `isSending`. Проверять конкретный `requestId`. Использовать `queueMicrotask` или `startTransition` для отложенного сброса.

2.2 Инфраструктура и DevOps

C-INFRA-01

Critical

Расположение: `docker/entrypoint.sh` (строки 16-23), `docker/nginx.conf` (строка 116), `docker/nginx-ssl.conf` (строка 120)

Суть ошибки: Оба шаблона `nginx` содержат `${PROXY_FETCH:-https://fetch.example.com}`, но `PROXY_FETCH` не включён в список `envsubst` в `entrypoint.sh` и не определён с `default`. Когда `envsubst` запускается с явным списком переменных, он подставляет только указанные -- остальное остаётся как буквальная строка. `Nginx` попытается распарсить `${PROXY_FETCH:-https://...}` как URI, что невалидно, и контейнер упадёт при запуске.

Риск: Production Docker-контейнеры не запускаются. Маршрут `/proxy/fetch/` полностью сломан.

Рекомендация: Добавить `PROXY_FETCH` в блок `defaults` и в аргументы `envsubst` в `entrypoint.sh`. Добавить строку : `"${PROXY_FETCH:-https://fetch.example.com}"` и включить в `export/envsubst`.

C-INFRA-02

Critical

Расположение: `.github/workflows/ci.yml` (строки 26, 48, 76, 95)

Суть ошибки: `Dockerfile` использует `npm ci --legacy-peer-deps` из-за конфликта `madge@8/typescript@6`. CI-пайплайн использует обычный `npm ci` во всех четырёх job. `npm v7+` отклоняет peer dependency конфликты по умолчанию, поэтому `npm ci` завершится ошибкой при установке зависимостей.

Риск: Каждый запуск CI падает на шаге `Install dependencies`. Ни `type-checking`, ни `linting`, ни `building`, ни `testing` никогда не выполняются. CI полностью нефункционален.

Рекомендация: Добавить `--legacy-peer-deps` ко всем вызовам `npm ci` в `ci.yml`, либо разрешить `peer-dep` конфликт обновлением `madge` или использованием `--force`.

2.3 Документация

C-DOC-01

Critical

Расположение: Разные файлы: `README.md` (v4.5.0), `docs/ПОЛНЫЙ_РЕЕСТР.md` (v4.6.0), `docs/STRUCTURE.md` (v4.6.0)

Суть ошибки: Два документа утверждают версию v4.6.0, в то время как все остальные указывают v4.5.0. В `CHANGELOG.md` нет записи о v4.6.0. Это создаёт противоречие о текущем состоянии проекта.

Риск: Неверные предположения о доступных функциях и состоянии развёртывания. Разработчики и AI-агенты могут полагаться на неверную версию при принятии архитектурных решений.

Рекомендация: Либо добавить запись v4.6.0 в `CHANGELOG`, либо откатить эти два файла к v4.5.0.

C-DOC-02

Critical

Расположение: Разные файлы: `README.md` (66+), `SINGLE_SOURCE.md` (64), `.ai_context.md` (36+), фактически 73

Суть ошибки: `SINGLE_SOURCE.md` -- предполагаемый канонический источник -- утверждает 64 контракта, хотя фактически 73 файла (64 top-level + 9 storage). `.ai_context.md` указывает 36+, что сильно устарело. `CHANGELOG` v4.1.0 указывает 32.

Риск: Разработчики и AI-агенты полагаются на эти числа для понимания архитектуры. Ошибочные подсчёты подрывают доверие ко всей документации.

Рекомендация: Обновить `SINGLE_SOURCE.md` до 73 (или 64+9). Распространить корректное число на `README.md`, `AGENTS.md`, `STRUCTURE.md`.

C-DOC-03

Critical

Расположение: `docs/SINGLE_SOURCE.md` (строка 23)

Суть ошибки: Утверждает "Circular deps: Нет", но `DEBT_REPORT.md` документирует 19 известных циклических зависимостей в `kernel`. Скрипт `npm run check:circular-kernel` завершается с `exit code 1`.

Риск: Вводящее в заблуждение утверждение может привести к тому, что разработчики будут считать кодовую базу чистой и пропускать проверки на циклические зависимости.

Рекомендация: Изменить на "Circular deps: 19 known (kernel)" или добавить предупреждение.

C-DOC-04

Critical

Расположение: `docs/SINGLE_SOURCE.md` (строка 25)

Суть ошибки: Утверждает "Inline styles: Нет" (100% через `common.ts`), но `debateusability.md` документирует, что `DebatePanel` содержит ~95% инлайн-стилей (1207 строк). Утверждение ложно.

Риск: Разработчики могут предполагать, что инлайн-стили устранены, и принимать неверные решения о рефакторинге.

Рекомендация: Изменить на "Inline styles: Partial" и добавить примечание о том, что некоторые крупные панели всё ещё используют инлайн-стили.

2.4 Тестирование

C-TEST-01

Critical

Расположение: e2e/basic-flow.spec.ts

Суть ошибки: Только 4 E2E теста, все используют очень свободные regex-матчеры (/key management|api key/i, /agent|builder/i, /type|message|send/i), которые могут совпасть почти с чем угодно на странице. Эти тесты будут давать ложноположительные результаты -- они могут проходить даже при сломанной функциональности.

Риск: E2E тесты -- последняя линия защиты. С свободными матчерами они не обеспечивают реальной защиты. Баги могут пройти незамеченными.

Рекомендация: Затянуть все текстовые матчеры до точных строк. Добавить тесты с взаимодействием (ввести текст, нажать submit, проверить появление ответа). Добавить проверки реального потока данных, а не только видимости элементов.

C-TEST-02

Critical

Расположение: vitest.config.ts, package.json

Суть ошибки: Конфигурация Vitest не содержит секции coverage. Нет скрипта coverage в package.json. Пороги покрытия не установлены. @vitest/coverage-v8 не установлен.

Риск: Без измерения покрытия пробелы в тестировании невидимы. Нет CI-приврата, предотвращающего поставку протестированного кода.

Рекомендация: Добавить @vitest/coverage-v8 в devDependencies, добавить секцию coverage в vitest.config.ts с порогами (lines: 60, branches: 50, functions: 50), добавить скрипт test:coverage в package.json.

C-TEST-03

Critical

Расположение: src/stores/ (6 файлов stores, 0 тестов)

Суть ошибки: Ни один из 6 Zustand stores (useKeyStore, useChatStore, useSystemStatus, useKeyIntelligence, debateLiveStore, topologyTraceStore) не имеет тестовых файлов. Stores содержат ключевое состояние приложения (ключи, сессии чата, статус системы) и импортируются почти каждым компонентом.

Риск: Баг в useKeyStore или useChatStore каскадно распространится на все компоненты. Логика stores -- основа приложения, и её некорректная работа критична.

Рекомендация: Создать тестовые файлы для каждого store, особенно useKeyStore.test.ts и useChatStore.test.ts, тестируя переходы состояний, действия и селекторы.

C-TEST-04

Critical

Расположение: src/hooks/ (6 файлов hooks, 0 тестов)

Суть ошибки: Ни один из 6 пользовательских хуков (`useMediaQuery`, `useAutoClearError`, `useKeyboardShortcut`, `useBookmarkShortcut`, `useConfirm`, `useTopicSuggester`) не имеет тестов. Хуки инкапсулируют переиспользуемую логику (горячие клавиши, очистка ошибок, диалоги подтверждения).

Риск: Баги в хуках затронут все использующие их компоненты. Регрессии могут пройти незамеченными.

Рекомендация: Добавить тесты на основе `renderHook` из `@testing-library/react`.

3. ВЫСОКИЕ ПРОБЛЕМЫ (High)

3.1 Исходный код

H-CODE-01

High

Расположение: `App.tsx` (строки 364, 439)

Суть ошибки: Дублирование `id="main-content"` -- и внешний, и внутренний имеют одинаковый ID. Ссылка `skip-nav` указывает на первый элемент (обёртку), а не на содержимое.

Риск: Нарушение HTML-спецификации, непредсказуемое поведение `accessibility`-инструментов.

Рекомендация: Удалить `id="main-content"` из внешнего, оставить только на .

H-CODE-02

High

Расположение: `App.tsx` (строка 362)

Суть ошибки: `GlobalErrorBoundary` с `key={location.pathname}` вызывает размонтирование и повторное монтирование всего дерева компонентов при каждой смене маршрута. Это уничтожает всё локальное состояние, отменяет все `pending effects` и заставляет `React` пересоздавать весь `DOM`.

Риск: Значительное падение производительности при навигации, потеря состояния компонентов, потенциальные утечки памяти от осиротевших подписок.

Рекомендация: Удалить `key` prop из `GlobalErrorBoundary`. `Error boundaries` должны сохраняться между сменами маршрутов. При необходимости изоляции -- оборачивать каждый `Route` отдельно.

H-CODE-03

High

Расположение: `stores/useSystemStatus.ts` (строки 46-52)

Суть ошибки: Второй `useEffect` имеет `[lastUpdated]` как зависимость. При каждом изменении `lastUpdated` (каждые ~30 секунд + по каждому событию) старый `interval` очищается и создаётся новый.

Риск: Неоправданные циклы `teardown/setup`, дрейф в точности измерения.

Рекомендация: Использовать `ref` для `lastUpdated` вместо включения в массив зависимостей.

H-CODE-04

High

Расположение: `kernel/events/event-bus.ts` (строки 343-357)

Суть ошибки: При `emitDepth > 16` событие откладывается через `setTimeout`. После 3 откладываний одно и то же событие навсегда удаляется. Это потерянный `circuit breaker` -- данные уничтожены.

Риск: В системе, полагающейся на события для синхронизации состояния (chat streaming, key state), потерянные события означают потерю данных -- сообщения чата могут никогда не завершиться.

Рекомендация: Вместо удаления -- ставить события в очередь и доставлять после раскрутки рекурсии. Или использовать microtask queue для последовательной обработки.

H-CODE-05

High

Расположение: kernel/security.ts (строки 83-135)

Суть ошибки: При reEncrypt, если перешифрование частично успешно, но сохранение salt не удаётся (например, storage full), старый salt уже перезаписан initialize(), а новый не сохранён -- хранилище в невозстановимом состоянии.

Риск: Пользователь может быть навсегда заблокирован от своих зашифрованных данных.

Рекомендация: Сохранять новый salt ДО начала перешифрования (с пометкой "pending"), подтверждать после успеха, откатывать к старому salt при неудаче.

H-CODE-06

High

Расположение: stores/debateLiveStore.ts, topologyTraceStore.ts

Суть ошибки: Оба Zustand stores регистрируют подписки на события и setInterval внутри create(). Метод destroy() существует для очистки, но ни один компонент или lifecycle hook никогда его не вызывает.

Риск: Накопление подписок и таймеров, утечки памяти, дублирование обработки событий.

Рекомендация: Вызывать destroy() из useEffect cleanup или из последовательности завершения runtime.

H-CODE-07

High

Расположение: main.tsx (строки 30-58)

Суть ошибки: При наличии #reset в URL приложение автоматически удаляет ВСЕ API-ключи и добавляет замены из hardcoded массива. Это происходит без какого-либо подтверждения пользователя.

Риск: Пользователь, перейдя по ссылке с #reset (из закладки, shared link или истории браузера), потеряет все свои ключи без предупреждения.

Рекомендация: Добавить window.confirm() перед удалением ключей или требовать специальный токен/секрет в hash для авторизации сброса.

H-CODE-08

High

Расположение: stores/useKeyStore.ts (строки 299-315)

Суть ошибки: Функция importKeys парсит JSON с reviver, защищающим от prototype pollution, но не проводит валидацию схемы. Весь объект item (включая любые лишние свойства) передаётся в groupManager.createKey().

Риск: Малформированные или вредоносные данные импорта могут внедрить непредвиденные свойства в путь создания ключа, вызывая ошибки или повреждение данных.

Рекомендация: Определить строгую схему для импортируемых ключевых объектов и валидировать/удалять неизвестные свойства перед передачей в createKey.

3.2 Инфраструктура и DevOps

Н-INFRA-01

High

Расположение: Dockerfile (строка 20) vs ci.yml (строка 10)

Суть ошибки: Dockerfile использует node:20-alpine, а CI использует NODE_VERSION: 22. TypeScript 6.x и Vite 8.x могут вести себя по-разному на разных мажорных версиях Node.

Риск: Сборка может проходить в CI, но падать в Docker (или наоборот). Сублимные различия в runtime между тестовой и production-средой.

Рекомендация: Синхронизировать обе среды на одну мажорную версию Node.

Н-INFRA-02

High

Расположение: docker/nginx.conf (строка 21) vs docker/nginx-ssl.conf (строка 41)

Суть ошибки: HTTP (dev) CSP использует connect-src "self" https: wss:, что разрешает подключения к ЛЮБОМУ HTTPS домену. HTTPS (prod) CSP ограничивает connect-src конкретными доменами провайдеров. В CSP для dev разрешены все HTTPS-соединения.

Риск: В скомпрометированной dev-среде возможно эксфильтрация данных на любой HTTPS endpoint.

Рекомендация: Синхронизировать dev CSP с prod или заменить https: на явный список доменов.

Н-INFRA-03

High

Расположение: scripts/cors-proxy.mjs (строки 57-124)

Суть ошибки: Прокси устанавливает Access-Control-Allow-Methods: GET, POST, OPTIONS, но: (1) не обрабатывает OPTIONS-запросы напрямую -- пересылает их целевому серверу; (2) использует t client.get(), который отправляет только GET-запросы. POST-запросы молча конвертируются в GET.

Риск: Browser-based POST-запросы через этот прокси молча не работают. Любой API-вызов, требующий POST, сломан.

Рекомендация: Добавить явную обработку OPTIONS и использовать client.request() с method forwarding.

Н-INFRA-04

High

Расположение: docker-compose.yml

Суть ошибки: Не определены mem_limit, cpus, pids_limit или deploy.resources.limits. Случайный процесс или утечка памяти в контейнере может потребить все ресурсы хоста.

Риск: Контейнер может вызвать OOM-kill процессов хоста, отказ в обслуживании или дестабилизацию соседних сервисов.

Рекомендация: Добавить deploy.resources.limits с cpus и memory.

Н-INFRA-05

High

Расположение: docker/nginx.conf, docker/nginx-ssl.conf

Суть ошибки: Все /proxy/* и /api/ location блоки не имеют rate limiting. Атакующий или багованный клиент может затопить API провайдеров LLM, быстро сжигая кредиты.

Риск: Неконтролируемый расход API-средств, потенциальные баны IP от провайдеров.

Рекомендация: Добавить `limit_req_zone` и `limit_req` директивы в `nginx` конфиг.

3.3 Документация

Н-DOC-01

High

Расположение: README.md (20), 02-core-concepts.md (25), CHANGELOG v4.5.0 (22 nodes)

Суть ошибки: Документ `core-concepts` утверждает 25 агентов, все остальные источники -- 20. Топология содержит 22 узла, включая `router/aggregator`.

Риск: Разработчик, создающий новую функцию, может спроектировать для неверного количества агентов.

Рекомендация: Стандартизировать на "20 debate agents" во всех документах. Уточнить, что 25 включает 5 документационных агентов другой категории.

Н-DOC-02

High

Расположение: README.md vs `.superagents/ARCHITECTURE.md`

Суть ошибки: README показывает "Legacy Service Layer" (thin Proxy wrappers). ARCHITECTURE.md описывает "Service Layer" (business logic, orchestration). Это принципиально разные архитектуры.

Риск: Разработчик, читающий ARCHITECTURE.md, будет предполагать, что сервисы содержат бизнес-логику, нарушая реальное архитектурное ограничение.

Рекомендация: Обновить ARCHITECTURE.md, отразив реальный слой: "Legacy Service Layer -- thin Proxy wrappers (delegate to kernel, no business logic)".

Н-DOC-03

High

Расположение: docs/DAL_PLAN.md

Суть ошибки: План рекомендует "Вариант С" (lightweight -- добавить getters к DatabaseService), но `src/kernel/dal/` содержит 11 файлов, реализующих полные DAL repositories (Вариант А). План сейчас вводит в заблуждение.

Риск: Разработчики могут думать, что DAL не построен, или попытаться реализовать Вариант С поверх существующего Варианта А.

Рекомендация: Обновить DAL_PLAN.md, отразив, что Вариант А был реализован, добавить заметки о реализации.

Н-DOC-04

High

Расположение: docs/README.md (строка 47), docs/STRUCTURE.md (строка 13)

Суть ошибки: docs/README.md ссылается на docs/architecture.md, который не существует (реальный файл -- docs/01-system-architecture.md). docs/STRUCTURE.md ссылается на fixtask.md, который не существует нигде в проекте.

Риск: Сломанные ссылки в документации приводят к 404.

Рекомендация: Обновить ссылки на реально существующие файлы или удалить.

Н-DOC-05

High

Расположение: docs/01-system-architecture.md (строка 72)

Суть ошибки: Обе EN/RU версии указывают src/kernel/service-registration.ts как ключевой файл. Реальное расположение -- src/kernel/service-registration/index.ts (директория).

Риск: Разработчики не найдут файл по документированному пути.

Рекомендация: Обновить на src/kernel/service-registration/ (директория).

Н-DOC-06

High

Расположение: SINGLE_SOURCE.md (~90), .ai_context.md (57), STRUCTURE.md (55+)

Суть ошибки: Три источника указывают разное количество тестов: ~90, 57, 55+. Фактическое количество -- 45 тестовых файлов. Все три источника переоценивают.

Риск: Тестовое покрытие кажется больше, чем в реальности. Ложное чувство безопасности.

Рекомендация: Обновить SINGLE_SOURCE.md на фактическое количество, распространить на остальные документы.

Н-DOC-07

High

Расположение: docs/COGNITIVE_RUNTIME_SPEC.md

Суть ошибки: Файл написан полностью на русском, тогда как все остальные пронумерованные документы (00-10) имеют отдельные EN и RU версии. Это нарушает конвенцию разделения языков.

Риск: Англоязычные участники не могут прочесть этот spec без перевода.

Рекомендация: Создать английскую версию или разделить на EN/RU пару.

Н-DOC-08

High

Расположение: docs/SYSTEM_PASSPORT.md vs docs/03-cognitive-layers.md

Суть ошибки: SYSTEM_PASSPORT перечисляет 5 когнитивных слоёв как: Execution, Coordination, Reasoning, Observability, Interpretation. Документ 03 определяет их как: Generation, Control, Diversity, Measurement, Interpretation. Это полностью разные имена и порядок.

Риск: Два авторитетных документа определяют когнитивную архитектуру системы по-разному.

Рекомендация: Синхронизировать SYSTEM_PASSPORT с каноническим определением из docs/03.

3.4 Тестирование

Н-TEST-01

High

Расположение: src/services/ChatService.test.ts

Суть ошибки: Тест импортирует реальный eventBus singleton и не мокирует его. Третий тест вызывает chatService.destroy() на реальном singleton. Это загрязняет глобальный event bus.

Риск: Другие тесты, выполняющиеся параллельно, могут получать чужие события. destroy() на real singleton может сломать другие тесты. Изоляция тестов нарушена.

Рекомендация: Мокировать eventBus, как это делает ChatService.autoRouting.test.ts.

Н-TEST-02

High

Расположение: Почти все компонентные тесты

Суть ошибки: Тесты используют expect(screen.getByText("X")).toBeDefined(). Однако toBeDefined() только проверяет, что значение не undefined. getByText бросает исключение, если элемент не найден, так что toBeDefined() всегда проходит -- это no-op assertion.

Риск: Сотни ложноположительных утверждений. Тесты проходят даже тогда, когда проверяемая функциональность сломана.

Рекомендация: Заменить все toBeDefined() на toBeInTheDocument() из @testing-library/jest-dom.

Н-TEST-03

High

Расположение: src/tests/setup.ts (строки 79-81)

Суть ошибки: Setup file импортирует и запускает весь runtime: import { runtime } и await runtime.start(). Это означает, что каждый тестовый файл загружает полный DI-контейнер и инициализирует все сервисы.

Риск: Медленный запуск тестов, побочные эффекты между тестами из-за общего состояния сервисов, невозможность тестировать сервисы изолированно.

Рекомендация: Переместить runtime bootstrap в явный beforeAll в тестах, которым он нужен. Позволить отдельным тестам opt-in.

Н-TEST-04

High

Расположение: src/kernel/services/ (251 файл, 8 тестов)

Суть ошибки: Только 8 из ~251 файлов kernel services имеют тесты. Среди непротестированных критических сервисов: key-service, debate-service, provider-router, health-service, budget-service, cache-service, settings-service, database-service.

Риск: Ключевая бизнес-логика (управление ключами, дебаты, маршрутизация провайдеров, бюджет, кэширование) не покрыта тестами.

Рекомендация: Приоритизировать тесты для key-service, debate-service, health-service, budget-service, cache-service.

Н-TEST-05

High

Расположение: Компонентные тесты (ToolsPanel, DashboardPanel, и др.)

Суть ошибки: Почти все компонентные тесты используют динамические import (await import()) внутри каждого it() блока для получения "свежих" экземпляров. Однако Vitest поднимает vi.mock(), так что динамический import не сбрасывает состояние модуля. Паттерн неэффективен и создаёт иллюзию изоляции.

Риск: Непоследовательные паттерны, более медленные тесты, ложное чувство изоляции.

Рекомендация: Стандартизировать на статические import с vi.mock() и vi.clearAllMocks() в beforeEach.

4. ПРОБЛЕМЫ СРЕДНЕГО ПРИОРИТЕТА (Medium)

Ниже представлены наиболее значимые проблемы среднего приоритета. Полный список доступен в приложении к данному отчёту.

M-CODE-01 | `bridges/useRoutingIntelligence.ts`

`setConfig` выставлен как `raw React.setState` без валидации. Потребители могут установить любую конфигурацию маршрутизации без проверки.

M-CODE-02 | `route-registry.tsx`

60+ иконок создаются как JSX-элементы при загрузке модуля. Эти элементы никогда не собираются GC и увеличивают время парсинга бандла.

M-CODE-03 | `kernel/kernel.ts`

`validateState` не проводит глубокую валидацию объекта `providers` -- приводит весь объект без проверки структуры.

M-CODE-04 | `App.tsx` (строки 69-143)

`ChatPanel`, `DashboardPanel`, `ProviderManager` и ещё ~20 тяжёлых компонентов импортируются `eagerly`, а не через `React.lazy()`. Увеличивает `time-to-interactive`.

M-CODE-05 | `kernel/security.ts`

При `persist=false` `salt` хранится в `sessionStorage`, который переживает перезагрузку страницы. При повторной инициализации с другим паролем используется старый `salt`.

M-CODE-06 | `kernel/runtime.ts`

`shutdown()` не очищает `container` -- сервисы сохраняют ссылки на таймеры и подписки. Утечки памяти после `shutdown/restart` циклов.

M-INFRA-01 | `docker/entrypoint.sh`

`PROXY_GENERIC` определён в `entrypoint.sh`, но не используется ни в одном `nginx`-шаблоне. Мёртвая конфигурация.

M-INFRA-02 | `package.json`

`vite-plugin-wasm` в `dependencies` (не `devDependencies`), но нигде не импортируется. Постоянно устанавливается в `production`.

M-INFRA-03 | .github/workflows/ci.yml

CI не включает шаг Docker build/test. Критический баг PROXY_FETCH (C-INFRA-01) не был бы обнаружен CI.

M-INFRA-04 | monitor-playwright.js (строка 56)

Баг: if (state.kids === 0) icon + " emoji" -- результат конкатенации отбрасывается. Должно быть icon += " emoji".

M-INFRA-05 | Dockerfile

Floating minor-version теги (node:20-alpine, nginx:1.27-alpine). Пересборка завтра может вытянуть другой patch, нарушая воспроизводимость.

M-INFRA-06 | scripts/cors-proxy.mjs

Прокси буферизует весь ответ в памяти (body array + Buffer.concat). Для больших LLM streaming-ответов это ломает SSE и может вызвать spike памяти.

M-DOC-01 | docs/00-overview.md

DebateService описан как "synchronous", но он использует async/await повсеместно. Вероятно имелся в виду "single-session" или "sequential".

M-DOC-02 | SYSTEM_MANIFEST.md EN vs RU

EN и RU версии расходятся по содержанию. RU более детальный и актуальный, включает разделы, отсутствующие в EN.

M-DOC-03 | CHANGELOG_RU.md

RU changelog не содержит записей для v4.4.2, v4.2.3, v4.2.2, v4.2.1, v4.1.0, v4.0.x, v3.7.x, v3.6.0, v3.5.x. Также содержит v3.1.0, v3.0.0, v2.4.0, отсутствующие в EN.

M-DOC-04 | docs/schema.sql

SQL-схема не содержит таблиц debate_sessions и debate_verdicts, которые документированы в STATE_ARCHITECTURE_AUDIT.md.

M-DOC-05 | docs/DEBT_REPORT.md

Не включает 5 pre-existing test failures, задокументированных в .ai_context.md (DebateService 8/18, SettingsService 5/8, и др.).

M-DOC-06 | UI Panel count

S README (47+), SINGLE_SOURCE (75+), docs/README (50+), STRUCTURE (75+), .ai_context (7/21 tested). Фактически 83 директории компонентов. Шесть разных чисел.

M-TEST-01 | e2e/playwright.config.ts

Нет reporter, нет screenshot/video on failure, нет health check URL для webServer. Недостаточная диагностика при падении E2E.

M-TEST-02 | src/core/TaskQueue.test.ts

Тест конкурентности использует setTimeout(r, 50) без vi.useFakeTimers(). Потенциально flaky в медленном CI.

M-TEST-03 | src/kernel/services/config-history.test.ts

Напрямую мутирует глобальный CONFIG объект (CONFIG.llm.retry.maxRetries = 99). Если тест упадёт посередине, глобал corrupted.

M-TEST-04 | LLM adapters

Только gemini-adapter.test.ts существует. Остальные 5+ адаптеров (openrouter, cerebras, nvidia, cloudflare, mock) не протестированы.

M-TEST-05 | LLM decorators

Только cache-decorator.test.ts. Остальные 11 декораторов (retry, fallback, circuit-breaker, rate-limit, и др.) не протестированы.

5. НИЗКОПРИОРИТЕТНЫЕ ПРОБЛЕМЫ (Low)

Ниже перечислены проблемы низкого приоритета. Они не представляют немедленной угрозы, но влияют на поддерживаемость, консистентность и качество кода.

L-CODE-01: route-registry.tsx: Aliased imports (Zap as TournamentZap) затрудняют поиск.

L-CODE-02: debateLiveStore.ts: currentThinking map keying -- sessionId вместо sessionId:agentId.

L-CODE-03: data/role-library.ts: BOM-символ в начале файла.

L-CODE-04: App.tsx: Инлайн-стили повсюду вместо дизайн-системы.

L-INFRA-01: nginx.conf: X-XSS-Protection header отсутствует в HTTP, но есть в HTTPS.

L-INFRA-02: .env.example: SYNC_SECRET= с пустым значением может ввести в заблуждение.

L-INFRA-03: nginx.conf в корне проекта: LEGACY, но всё ещё существует и может запутать.

L-INFRA-04: Отсутствует .prettiignore -- Prettier может пытаться форматировать dist/ и coverage/.

L-DOC-01: debateusability.md: Содержит сырой вывод AI-сессии без форматирования.

L-DOC-02: SYSTEM_PASSPORT.md: Утверждает "Backend: Node.js Runtime", но система -- browser-only SPA.

L-DOC-03: SYSTEM_PASSPORT.md: Утверждает "WebSocket Streaming", но используется SSE.

L-DOC-04: Отсутствуют RU-переводы для 8 ключевых документов.

L-TEST-01: Нет test:watch скрипта в package.json.

L-TEST-02: Нет shared test utilities (mock factories, custom render).

L-TEST-03: console.log остался в cache-decorator.test.ts.

6. АРХИТЕКТУРНЫЕ ПРОБЛЕМЫ

Смешанные паттерны управления состоянием

Проект одновременно использует три различных подхода: (1) Zustand (useChatStore, debateLiveStore, topologyTraceStore), (2) useSyncExternalStore (useKeyStore), (3) useState + eventBus (useSystemStatus, usePoolStatus, useRoutingIntelligence). Это создаёт когнитивную нагрузку для разработчиков и непоследовательные паттерны для тестирования, очистки и гидратации. Рекомендация: стандартизировать на одном подходе (предпочтительно Zustand с subscribeWithSelector для производных состояний).

Service Locator антипаттерн через Proxy Resolver

Функция resolve() в kernel/resolver.ts реализует service locator через JavaScript Proxy. Хотя это позволяет отложенную инициализацию, она: делает зависимости невидимыми (нет constructor injection), скрывает ошибки до момента обращения к свойствам runtime, затрудняет тестирование (нельзя мокировать при конструировании). Рекомендация: мигрировать на явный DI с constructor injection или хотя бы добавить типизированные интерфейсы для всех сервисов.

EventBus как основной канал коммуникации

Приложение использует EventBus для почти всей межмодульной коммуникации, включая синхронизацию состояния. Это создаёт: скрытый поток данных (трудно отследить, откуда происходят изменения состояния), проблемы eventual consistency (обновления через события асинхронны), сложности тестирования (необходимо подписываться на события для проверки поведения). Рекомендация: дополнить EventBus механизмом трассировки (event provenance log) и рассмотреть переход на более явные паттерны для критических потоков данных.

7. ПРОБЕЛЫ В ТЕСТОВОМ ПОКРЫТИИ

Module	Source Files	Test Files	Est. Line Coverage
Components	~60	28	~35%
Kernel Services	251	8	<5%

Stores	6	0	0%
Hooks	6	0	0%
LLM Adapters	~15	1	~7%
LLM Decorators	12	1	~8%
LLM Core	5	2	~40%
Utils	7	0	0%
Overall	703	45	~10-12%

Критические пути без тестов:

- Жизненный цикл ключей (добавление, ротация, отзыв, компрометация API-ключей end-to-end)
- Поток сообщений чата (send > route > LLM call > stream response > update store)
- Маршрутизация провайдеров (auto-routing решения, fallback chains, circuit breaker)
- Оркестрация дебатов (start > rounds > consensus > verdict)
- Сохранение данных (save/load из IndexedDB и SQLite)
- Контроль бюджета (spend tracking, quota enforcement, budget alerts)
- Безопасность (key vault encryption, compromise detection, key rotation)
- Event sourcing (event recording, checkpointing, replay)

8. ЧЕК-ЛИСТ САМОПРОВЕРКИ

Перед исправлением проблем рекомендуется использовать следующий чек-лист для верификации каждого исправления:

Критические баги (Critical)

- ☐ Resolver возвращает безопасные fallback для всех 40+ сервисов?
- ☐ API-ключи больше не логируются в консоль?
- ☐ useChatStore использует индексированный поиск?
- ☐ sendMessage использует Set вместо булева флага?
- ☐ endpoint.sh включает PROXY_FETCH в envsubst?
- ☐ CI использует --legacy-peer-deps?
- ☐ SINGLE_SOURCE.md содержит корректные числа?
- ☐ Версия унифицирована во всех документах?

Высокие проблемы (High)

- ☐ GlobalErrorBoundary больше не имеет key={location.pathname}?

- ☐ `destroy()` вызывается для `debateLiveStore` и `topologyTraceStore`?
- ☐ `#reset` требует подтверждения пользователя?
- ☐ Node.js версии синхронизированы между Docker и CI?
- ☐ CSP в dev-среде ограничивает `connect-src`?
- ☐ CORS-прокси обрабатывает `OPTIONS` и пересылает `method`?
- ☐ `EventBus` не удаляет события навсегда?
- ☐ `changePassword` использует `two-phase commit`?
- ☐ Количество агентов унифицировано в документации?

Тестирование

- ☐ Покрытие кода измеряется и имеет пороги?
- ☐ Stores покрыты модульными тестами?
- ☐ assertions используют `toBeInTheDocument()` вместо `toBeDefined()`?
- ☐ E2E тесты используют точные матчеры?
- ☐ `Runtime bootstrap` опционален в `setup.ts`?
- ☐ `Kernel services` имеют приоритетные тесты?

Документация

- ☐ Все ссылки указывают на существующие файлы?
- ☐ EN/RU версии синхронизированы?
- ☐ Счётчики (панели, контракты, тесты) верны во всех документах?
- ☐ `DAL_PLAN` отражает реальную реализацию?
- ☐ Архитектурные диаграммы согласованы между документами?

9. ИТОГОВАЯ ОЦЕНКА КАЧЕСТВА

Критерий	Оценка (1-10)	Комментарий
Архитектура	6	Концептуально здоровая, но смешанные паттерны состояния и <code>service locator</code> снижают оценку
Безопасность	4	Утечка API-ключей, <code>#reset</code> без подтверждения, слабая XOR-обфускация, невалидированный импорт
Надёжность	4	Потеря событий в <code>EventBus</code> , <code>race conditions</code> , невозможное хранилище при ошибках
Тестирование	3	~10-12% покрытие, 0 тестов для <code>stores/hooks</code> , ложноположительные assertions
Инфраструктура	4	Docker не запускается (<code>PROXY_FETCH</code>), CI сломан, нет <code>rate limiting</code>

Документация	5	Обширная, но с противоречиями, неверными счётчиками и сломанными ссылками
Код-стайл/поддерживаемость	6	TypeScript, ESLint, Prettier, но инлайн-стили и 80+ маршрутов в App.tsx
Производительность	5	$O(n*m)$ при streaming, eager imports, JSON.stringify для equality

ИТОГОВАЯ ОЦЕНКА: 4.6 / 10

Проект имеет интересную архитектурную концепцию, но страдает от системных проблем в ключевых областях: критические баги в инфраструктуре (Docker не запускается, CI сломан), утечка учётных данных, неудовлетворительное тестовое покрытие и противоречивая документация. Приоритеты исправления: (1) починить Docker/CI -- без этого невозможно развёртывание; (2) устранить утечку API-ключей; (3) исправить resolver и race conditions; (4) добавить тестовое покрытие для stores и kernel services; (5) синхронизировать документацию.

10. МАТРИЦА ПРИОРИТЕТОВ ИСПРАВЛЕНИЙ

Priority	Action	Effort	Impact
P0	Починить PROXY_FETCH в endpoint.sh	Low	Critical
P0	Добавить --legacy-peer-deps в CI	Low	Critical
P0	Устранить утечку API-ключей из консоли	Low	Critical
P0	Исправить resolver (safe fallback для всех сервисов)	Medium	Critical
P1	Рефакторинг useChatStore (indexed lookups)	Medium	High
P1	Удалить key из GlobalErrorBoundary	Low	High
P1	Добавить coverage tooling и пороги	Low	High
P1	Исправить false-positive assertions (toBeDefined)	Medium	High
P2	Синхронизировать SINGLE_SOURCE.md	Low	High
P2	Добавить тесты для stores и kernel services	Medium	High
P2	Синхронизировать Node.js версии Docker/CI	Low	Medium
P2	Добавить rate limiting в nginx	Low	Medium